

repo-wizard

AI agent-driven repo improvement analysis tool

Overview

Repo Wizard is a production-grade, local-first repository governance and code-hardening tool built on the Google Agent Development Kit (@google/adk).

Repo-wizard orchestrates a decoupled network of 31 specialized expert AI subagents to audit codebase security, resilience, and regulatory compliance entirely within a secure in-memory developer sandbox. All agent and orchestration code was written from scratch for this project.

Repo-wizard creates reports with coding practices, tooling suggestions, and other improvements that can assist vibe coding developer with little to no software development experience to build a high quality codebase.

By translating abstract, complex engineering best practices into clear, actionable, and bite-sized improvements, repo-wizard enables developers to develop robust applications without sacrificing the velocity of modern, AI-assisted development.

repo-wizard
approachable improvements
that adapt to where you want to be

Course Capstone Project

This was created as the capstone project at the end of the *AI Agents: Intensive Vibe Coding Course* presented by Google and Kaggle.

<https://www.kaggle.com/competitions/vibecoding-agents-capstone-project>

You can find the project summary report, including a quick demo video showing repo-wizard in action, here:

<https://www.kaggle.com/competitions/vibecoding-agents-capstone-project/writeups/new-writeup-1783403353889>

Core Capabilities

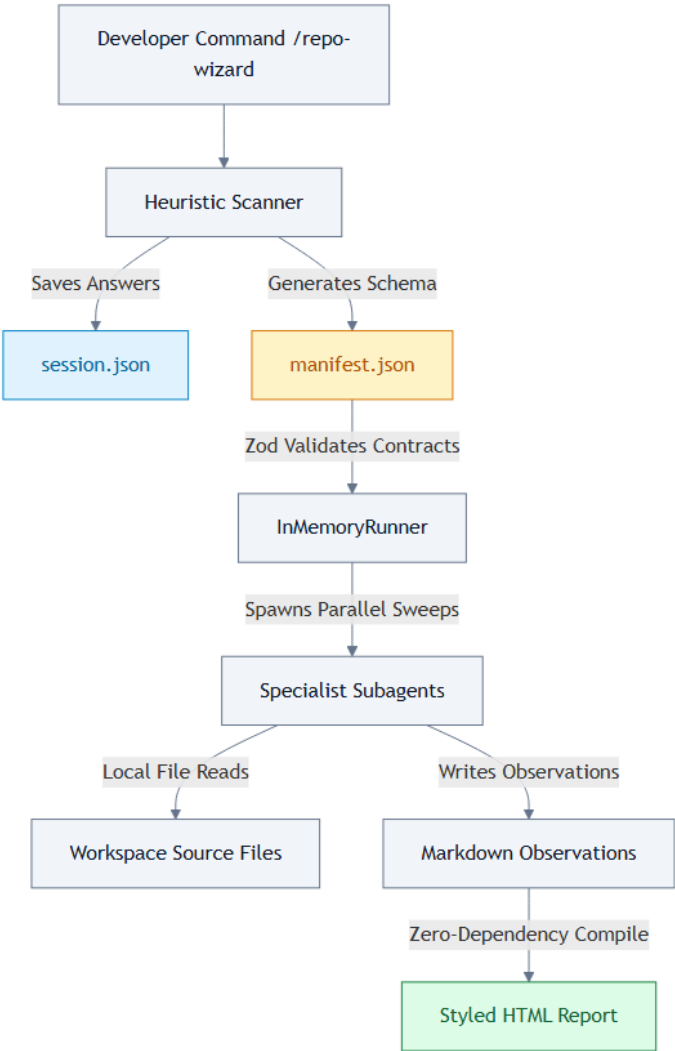
- **31-Specialist Swarm Audit:** Automatically distributes deep inspection tasks across 31 expert personas targeting security vulnerabilities, performance regressions, API contracts, and formal state verification.
- **Local-First Sandbox Security:** Resolves absolute file paths locally and executes in-memory, mitigating codebase exposure risks by keeping source content within host boundaries.
- **Interactive Stack Adaptation:** Conducts contextual onboarding questionnaires to dynamically generate custom linter, testing, and compliance presets.
- **Automated Verification Gates:** Synthesizes configurations, runs validation checks, and triggers Git rollback safety routines on tool setup failure.

System Architecture & Dataflows

The application avoids context-token bloat and non-deterministic agent behavior by implementing a decoupled, contract-driven Hybrid Orchestration Model (see image at right).

Though many specialized agents may be identified as relevant for the repo being examined, only 6 agents will be run at a time in order to reduce the likelihood of API throttling or running out of tokens.

- 1. **Mapping:** The developer triggers the scanner script, which indexes the directory structure, size metrics, and inferred languages to compile *session.json* and *manifest.json*
- 2. **Contracts:** The orchestrator validates parameter contracts using strict Zod runtime schemas before launching the ADK InMemoryRunner.
- 3. **Execution:** Parallel subagents receive isolated, absolute pathways. They pull file contents locally via sandboxed path-traversal tools, storing raw markdown observation blocks.
- 4. **Synthesis:** Zero-dependency compilers merge observation logs into consolidated Markdown summaries and responsive HTML deliverables.



If the user wants to ask questions about the tool suggstions, or update their project goals, they are always able to work with the team of agents to better understand where they are and what they need to do next.

Primary Tech Stack Summary

Category	Technology / Tool	Architectural Purpose
Language	TypeScript / Node.js	Runtime language environment providing structural typing.
Agent Orchestration	@google/adk	Manages InMemoryRunner, LlmAgent sessions, and native tool hooks.
Schema Validation	zod	Enforces structural constraints over manifest and questionnaire schemas.
State Storage	Local JSON Files	Houses resumable developer state and agent contract mappings.
Sandbox Security	Regex & Zod transforms	Strips null-byte (\0) injections and filters path traversal attempts.
Reporting Engine	Native JS Compilers	Compiles markdown outputs to styled HTML without NPM package bloat.

Quality Assurance & Verification

- **Linters Gates:** Comprehensive local validator scripts check agent configurations, command mappings, skill structures, and documentation formatting prior to merging changes.
- **E2E Testing Suite:** Dynamic E2E integration test harnesses spin up temporary execution environments to mock runner operations, verifying correct state transition paths.
- **Zero-Dependency Mandate:** Critical utilities, including the markdown compiler, are built natively using built-in Node.js libraries, protecting the developer environment from supply chain vulnerabilities.

Resilience, Security & Compliance

- **Exponential Backoff Retries:** The core orchestrator wraps LLM generation steps in automated 3-attempt exponential backoff retry loops, maintaining system tolerance during transient API failures.
- **Log Credential Redaction:** Outbound log interceptors parse text streams, programmatically scrubbing Git URLs, absolute directory paths, and API keys before outputting reports.
- **Path Traversal Shield:** Filesystem operations are routed through regular-expression filters and relative-path comparisons to isolate subagent sweeps within the active directory boundaries.

Developer Experience & Code Health

- **Session Resumability:** Decoupled state architecture allows developers to pause, resume, or progressively update sweeps across targeted pillars without losing historical progress.
- **Git Protections:** Onboarding scripts configure Husky hooks that execute automated pre-commit and pre-push validation loops, preventing non-compliant code from reaching repositories.
- **Prompt Parity Linting:** Automated checks verify agent prompts against token limits and dry-compliance guidelines, keeping developer swarms focused and cost-efficient.

Repo Guide

Private repo URL: <https://github.com/anjiera/repo-wizard>

To install the slash commands in Antigravity 2.0, run the **setup.ps1** or **setup.sh** script.

To inspect the architectural patterns and implementations:

- **runner.ts** : The central orchestration pipeline executing specialist agent lifecycles via ADK.
- **fs-tools.ts** : Sandboxed filesystem wrappers implementing path traversal protection and input sanitization.
- **initial-codebase-scan.js** : The static directory parser mapping languages, file paths, and metadata schemas.
- **adk-orchestration.md** : Technical specification detailing the local in-memory orchestration design decisions.

To find written descriptions of design decisions, and examples:

- **docs/design**: Design documents outlining high-level architecture
- **docs/AGENT_MATRIX.md** : a listing of all agents, and their metadata
- **docs/examples** : an example report made by running repo-wizard on itself

To read the agent code, view the **/agents** folder.

Screenshots

Sample output from running the Performance agents on the beat-the-heat repo, outlining the reports and artifacts generated by the team of agents.

The Repo Wizard audit has been successfully completed for the **Performance** pillar of the **beat-the-heat** repository!

Here is a summary of the generated deliverables:

- **Executive Summary** This report provides a high-level overview of the codebase's health, performance strengths, and areas of opportunity. Review this report to get a strategic overview of the recommended improvements.
- **Full Technical Report** This document details the complete technical profile of the codebase, including in-depth analysis from each specialist subagent and specific code modification blueprints. Refer to this file to obtain step-by-step implementation and integration guides for the telemetry and resilience improvements.
- **Observations List** This list consolidates all specific findings and localized code issues identified during the subagent scans. Use this as a reference checklist to track outstanding issues across the audited modules.
- **Backlog CSV** This file exports all recommended tasks in a structured format suitable for direct import into project management tools like Jira or GitHub Issues. Your next step should be importing these tickets to start planning your sprint.

Table of contents from the full technical report:

▼ Table of Contents

- 1. Executive Summary
- 2. Audit Scope & Environment Profile
 - 2.1 Environment Profile
 - 2.2 Audit Parameters
 - 2.3 Inferred Interview Profile
- 3. Maturity Model Guidance
- 4. Detailed Quality Pillars Analysis
 - Security & Compliance (N/A)
 - Performance & Resilience
 - Architecture & Design (N/A)
 - Code Quality & Testing (N/A)
- 5. Effort vs. Value Rollout Matrix
- 6. Conclusions

Sample section from the full technical report after running repo-wizard on an earlier version of the beat-the-heat repo:

Repo Wizard Executive Summary - beat-the-heat

▼ Section 1: Codebase Health & Strengths

This section highlights the key strengths of the target repository, detailing its modular structure, lightweight footprint, and predictable layout.

Overview: The target codebase exhibits a solid structural foundation, utilizing standard modular patterns and a predictable directory layout that facilitates navigation and comprehension. By leveraging native platform execution standards, the project maintains a lightweight build footprint, which is critical for local development efficiency.

Passive static analysis of the workspace indicates that core operations avoid complex pre-compilation steps that often slow down execution speed or complicate local environment configuration. The existence of script utilities indicates an active effort to automate repository tasks and maintain developer hygiene. This established pattern ensures that future modules can be added with zero disruption to the production runtime configurations. Furthermore, the codebase exhibits clean segregation of concerns, with script helpers, compile utilities, and CLI commands isolated in dedicated modules to protect the codebase from structural regressions during updates.

Language Profile: **Kotlin (Android application), Gradle Kotlin DSL configurations** (`build.gradle.kts` , `settings.gradle.kts`). **Deficiencies:** These logs are written to Android Logcat only. They lack structure (e.g., JSON syntax), trace correlation IDs, or centralized exporting capability. name: Weather Service Health Alerts

DataStore Multi-Instance Antipattern: The Jetpack DataStore preferences file is initialized multiple times across different repositories and Glance widget classes, presenting a high risk of thread lockups, write conflicts, and `IllegalStateException` crashes.

► Section 2: Tooling & Compliance Opportunities